

Airflow 3.x: XComs, TaskFlow API e Integração de APIs

Construindo um Pipeline ETL Completo

Disciplina de Extração de Dados

IBMEC

14 de maio de 2026

Objetivo da Aula

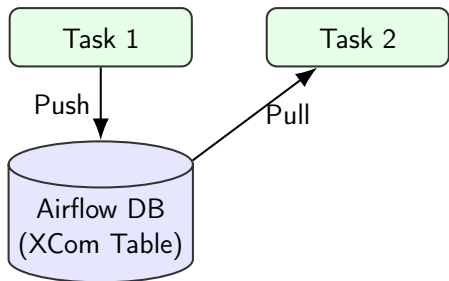
- Compreender a evolução arquitetural para o **Airflow 3.x** e a **TaskFlow API**.
- Aprender como tarefas compartilham dados entre si utilizando **XComs** (Cross-Communications).
- Extrair dados reais da internet utilizando requisições HTTP (APIs REST).
- Combinar Extração (API), Transformação (Pandas) e Carga (Banco de Dados) em um único Pipeline de Dados (ETL).

Revisão: O Problema do Isolamento de Tarefas



- Por design, as **Tasks** no Airflow são **independentes e isoladas**.
- Elas podem até rodar em servidores (Workers) diferentes!
- **Problema:** Se a Tarefa 1 extrai dados de uma API, como a Tarefa 2 consegue acessar esses dados para processar?
- Variáveis globais do Python **não funcionam** entre tarefas diferentes.

A Solução: XComs (Cross-Communications)



- **XComs** são mensagens serializadas (pequenos volumes de dados) que as tarefas trocam entre si.
- O dado é salvo temporariamente no **banco de dados interno do próprio Airflow**.
- A Tarefa 1 faz um Push (empurra o dado pro banco).
- A Tarefa 2 faz um Pull (puxa o dado do banco).
- **Limitação:** Não serve para transitar 10GB de dados! Apenas metadados, links, caminhos de arquivo ou JSONs pequenos.

O Paradigma Moderno: TaskFlow API (Airflow 3.x)

- Nas versões antigas do Airflow, criar XComs era verboso (`ti.xcom_push()` e `ti.xcom_pull()`).
- Com o **Airflow 3** e a **TaskFlow API**, o fluxo tornou-se nativo ao Python usando o decorador `@task`.
- Um simples `return` faz o Push. Passar o dado como argumento da função faz o Pull e define a dependência automaticamente!

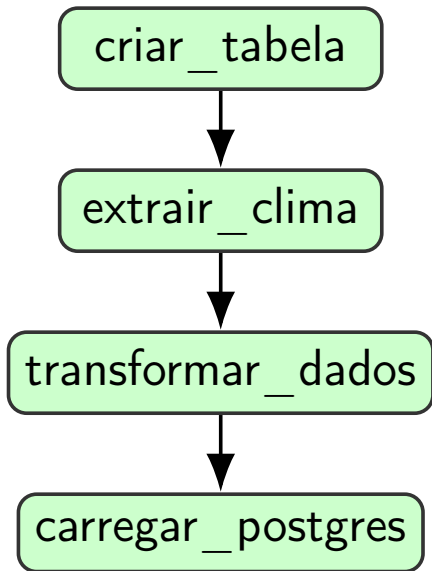
```
1 from airflow.decorators import task, dag
2 from datetime import datetime
3
4 @task
5 def extrair():
6     return {"cliente": "IBMEC", "status": "ativo"} # Isso eh um XCom Push!
7
8 @task
9 def transformar(dados): # Recebeu o argumento? Fez XCom Pull automático!
10     print(f"Processando {dados['cliente']}")
11
12 @dag(start_date=datetime(2026, 1, 1), schedule="@daily", catchup=False)
13 def meu_pipeline():
```

Consumindo APIs Reais

- O "E"(Extract) de ETL. Em vez de dados manuais, vamos acessar serviços vivos na web.
- Usaremos a biblioteca requests do Python.
- Exemplo: Consultar a previsão do tempo de Brasília pela API da Open-Meteo.

```
1 import requests
2
3 @task
4 def extrair_clima_api():
5     # Coordenadas de Brasilia
6     url = "https://api.open-meteo.com/v1/forecast?latitude=-15.78&longitude=-47.93&current_weather=true"
7
8     resposta = requests.get(url)
9
10    # Valida se a requisicao deu sucesso (Status 200)
11    resposta.raise_for_status()
12
13    dados = resposta.json()
14    # Retorna para o proximo passo do fluxo (XCom)
15    return dados['current_weather']
```

- Vamos unir o conhecimento de APIs, Pandas (Transformação) e Bancos de Dados (Carga da aula 16).
- Passos lógicos do nosso **Pipeline de Dados**:
 - 1 extrair_api: Pega o JSON do servidor web.
 - 2 transformar_dados: Converte o JSON num formato tabular e ajusta tipos.
 - 3 carregar_banco: Usa o `SQLExecuteQueryOperator` ou Hook para fazer um INSERT.



● Ação do Aluno:

- 1 Inicie o container do banco de dados (Postgres).
- 2 Execute a DAG `pipeline_etl_clima`.
- 3 Abra o Log da tarefa `extrair_clima` para ver o payload original da API.
- 4 Abra o painel de Administração do Airflow → **XComs** para ver os dados transitando entre as tarefas.
- 5 Abra seu DBeaver/PgAdmin para verificar os dados inseridos na tabela final!

- O Airflow 3 facilita drasticamente a Engenharia de Dados através da **TaskFlow API** (`@task`, `@dag`).
- Esqueça os `PythonOperators` verbosos de antigamente. Escreva código Python nativo.
- O Airflow gerencia os **XComs** e a **Ordem de Execução** (Grafo) através das passagens de parâmetros entre as funções.
- A separação de responsabilidades (E, T, L) em diferentes tarefas torna o pipeline robusto contra falhas (se a transformação falhar, não precisamos re-extrair da API).